OPENSTAGE

# Manuale tecnico e guida utente

Manuale tecnico e guida utente

**Versione:** 1.6

**Data:** 24 ottobre 2026

**Contenuto:** Fase 1 · Fase 1.2 · Fase 1.3 · Fase 2 · Fase 3 · Fase 4 · Fase 5.4 · Fase 5.5A · Fase 5.5B · Fase 5.5C · Fase 6 · Fase 7A

Documento generato automaticamente dai sorgenti in docs/manuale. Non modificare il PDF a mano: si aggiorna con npm run manuale.

# Indice


## 1. Che cos'è OpenStage

- A che punto siamo

## 2. I ruoli e le loro regole

- La regola importante: il ruolo è definitivo

## 3. Registrazione e primo accesso

- Con email e password
- Password dimenticata
- Con Google
- La barra in alto quando sei dentro
- Diventare artista o locale (solo se sei un fan)

## 4. Trovare un concerto

- La Home
- La mappa
- La ricerca
- I filtri rapidi
- Muro, lista e mappa
- Salvare un evento

## 5. Reclamare la tua scheda

- Artista
- Locale

## 6. Pubblicare una serata

## 7. Scrivere allo staff

## 8. Segnalare una serata

## 9. Navigare il sito

- Lingua
- Il player in basso (non ancora attivo)

## 10. Domande frequenti

## 11. Architettura e stack

- Alternative valutate e perché sono state scartate
- Versioni dell'ambiente

## 12. Requisiti, comandi e struttura del progetto

- Struttura delle cartelle

## 13. Variabili d'ambiente

## 14. Autenticazione e gestione del ruolo

- Come funziona l'accesso
- Dove sono applicate le regole del ruolo

## 15. Database

- Tabelle
- Funzioni e trigger
- Indici
- Migrazioni

## 16. Sicurezza: Row Level Security

- Avvisi di sicurezza attualmente accettati

## 17. Come si collauda la RLS

## 18. Internazionalizzazione

## 19. Design system

## 20. Mappe e tile

## 21. Ricerca, filtri e stato nell'URL

## 22. Deploy e rilascio

- Checklist di fine fase

## 23. Errori già incontrati e come si risolvono

## 24. Come si verifica una pagina (browser headless)

## 25. Il manuale PDF

## 26. Roadmap delle fasi

## 27. Diario delle fasi

- Fase 1 — Fondamenta
- Fase 1.2 — Indurimento del database
- Fase 1.3 — Ruoli definitivi
- Fase 2 — Scoperta degli eventi
- Fase 3 — Directory, reclami, editor e condivisione
- Fase 4 — Qualità dei dati e moderazione
- Fase 5.4 — Home poster wall
- Fase 6 — Fiducia pubblica e densità locale
- Fase 5.5A - Coerenza visiva: accesso e account
- Fase 7A - Email reale, conferma e recupero password
- Fase 5.5B - Scheda evento e popup mappa
- Fase 5.5C - Directory artisti e locali
- Come si aggiorna questo diario

## 28. Changelog del manuale

- Come si aggiorna

> OpenStage · Manuale tecnico e guida utente · versione 1.6 (24 ottobre 2026)
> Il PDF viene rigenerato dai file Markdown in `docs/manuale/` con `npm run manuale`.

# Che cos'è OpenStage

OpenStage è una piattaforma per **scoprire concerti dal vivo**, pensata per dare visibilità reale agli **artisti indipendenti** e ai **locali di zona** che non possono permettersi le grandi piattaforme commerciali.

I quattro principi che guidano ogni decisione (anche tecnica) sono:

| Principio | Cosa significa in pratica |
|---|---|
| **Underground e indipendente** | Gli artisti emergenti e i piccoli locali sono i protagonisti, non un contorno |
| **Gratis per sempre** | Nessun paywall per fan, artisti e locali nel piano base, nessun "pay-to-play" |
| **Autenticità** | Eventi reali, luoghi reali: niente metriche inventate o engagement finto |
| **UX senza attrito** | Trovare il concerto di stasera dev'essere facile come aprire una mappa |

> **Stato attuale.** Il progetto è in costruzione per fasi. Questo manuale documenta tutto ciò che è **realmente online e funzionante** alla versione indicata in copertina: quando una funzione è ancora un abbozzo, è scritto chiaramente.

## A che punto siamo

**Fatto e online:** identità e accesso (email e Google), **conferma email reale**, **recupero password**, **Turnstile verificato lato server**, ruoli con regole definitive, design system, doppia lingua IT/EN con indirizzi `/it` e `/en`, **directory reali di artisti e locali** con filtri e schede pubbliche, **reclamo della scheda**, **dashboard** per artisti e locali, **editor degli eventi** con bozze e locandine, **mappa interattiva** con pin e globo, **pagine evento** con calendario e condivisione, **ricerca con suggerimenti**, **filtri rapidi**, **pagina dei preferiti**, **segnalazioni e moderazione**, **Home poster wall** con **sezione Città**, **pagine pubbliche reali** (Chi siamo, Per gli artisti, Per le venue, Contatti, FAQ), **indice e pagine città**, **modulo contatti che scrive nel database** con coda messaggi per lo staff, database con sicurezza a livello di riga, deploy continuo.

**In arrivo nelle prossime fasi:** community e "segui", Spotify, pagamenti opzionali, testi legali definitivi e rifiniture SEO. La risposta integrata ai messaggi di contatto (`B48`) resta l'ultimo pezzo della 7A.

**Da sapere oggi:** gli eventi presenti sono **reali**. Possono essere inseriti dalla redazione oppure pubblicati direttamente da artisti e locali dopo il reclamo della scheda; finché nessuno dei due ha reclamato, la scheda evento lo dice. La Home mostra solo le serate in programma; le serate passate restano nelle pagine di artista, locale e nella dashboard di chi le ha pubblicate.

# I ruoli e le loro regole

Quando ti registri scegli **come vuoi usare OpenStage**. La scelta non è un'etichetta cosmetica: determina cosa puoi fare e quali pagine vedi.

| Ruolo | Per chi è | Cosa può fare |
|---|---|---|
| **Fan** | Chi vuole scoprire concerti | Esplorare mappa ed eventi, cercare, filtrare, salvare i preferiti |
| **Artista** | Musicisti, band, dj, progetti | Tutto ciò che fa un fan, più: profilo artista e pubblicazione dei propri concerti |
| **Locale / Organizzatore** | Locali, circoli, festival, promoter | Tutto ciò che fa un fan, più: profilo del locale e programmazione degli eventi |
| **Admin** | Solo lo staff di OpenStage | Accesso completo per assistenza e correzioni |

## La regola importante: il ruolo è definitivo

Questa è una scelta precisa del progetto, e vale **sia per la registrazione via email sia per quella con Google**:

| Situazione | Cosa succede |
|---|---|
| Ti registri come **Artista** | Resti artista **per sempre** |
| Ti registri come **Locale** | Resti locale **per sempre** |
| Ti registri come **Fan** | Puoi diventare **Artista o Locale**, ma **una volta sola**; dopo non si torna indietro |
| Sei un fan e vuoi tornare fan dopo essere diventato artista | Ti serve un **nuovo account** |

Il motivo è la coerenza della community: un profilo artista non deve poter diventare un locale (o viceversa) dall'oggi al domani, perché gli eventi, i fan e i locali che ci hanno interagito si fidano di quel profilo.

> **Se hai sbagliato a scegliere**, scrivi allo staff: l'admin può correggere il ruolo dalla dashboard (è l'unica eccezione prevista alla regola).

# Registrazione e primo accesso

## Con email e password

1. Vai su **Registrati** dalla barra in alto.
2. Inserisci nome, email e password.
3. Conferma il **captcha** (protezione anti-bot Cloudflare Turnstile).
4. Premi **Crea account**: riceverai un'email da `noreply@getopenstage.it`.
5. Apri il link di conferma. Dopo la conferma scegli il **ruolo** (Fan, Artista, Locale).
6. Se hai scelto **Artista** o **Locale**, compila la scheda del profilo; altrimenti torni alla Home da utente autenticato.

> La conferma email è **obbligatoria**: senza il link l'account non può accedere. Controlla anche la cartella spam. Il ruolo si sceglie **dopo** la conferma, non nel form di registrazione.

## Password dimenticata

1. Nella pagina **Accedi** clicca **Password dimenticata?**.
2. Inserisci l'email dell'account e completa il captcha.
3. Apri il link che arriva da `noreply@getopenstage.it`. Funziona anche se lo apri da un browser o da un dispositivo diverso da quello della richiesta.
4. Scegli la nuova password (almeno 8 caratteri) e salva.
5. Torna al login e accedi con la nuova password.

Se il link è scaduto, richiedine un altro dalla pagina **Password dimenticata?**.

## Con Google

1. Clicca **Continua con Google** nella pagina di accesso o registrazione.
2. Scegli il tuo account Google.
3. Google ha già verificato l'email: **non arriva nessuna email di conferma**.
4. Al primo ingresso compare la **schermata di scelta del ruolo**. Da quel momento la regola dei ruoli definitivi vale esattamente come per l'email.

## La barra in alto quando sei dentro

Dopo l'accesso, in alto a destra vedi:

- il tuo **indirizzo email**;
- un **badge colorato con il tuo ruolo** (viola = Artista, ciano = Locale, rosa = Admin, grigio = Fan);
- un **link contestuale**: per un fan "Diventa artista o locale", per un artista o un locale "Crea evento";
- il pulsante **Esci**.

## Diventare artista o locale (solo se sei un fan)

1. Clicca **Diventa artista o locale** nella barra in alto.
2. Scegli **Artista** o **Locale** e conferma.
3. Se scegli Artista vieni portato al profilo artista; il badge in alto cambia subito colore e il link di upgrade **sparisce**, perché la scelta è definitiva.

Se torni su quella pagina dopo la scelta, OpenStage ti mostra **"Il tuo ruolo è già attivo"** con il tuo ruolo: è la conferma che il blocco funziona anche lato database.

# Trovare un concerto

Questa è la parte che oggi funziona di più: la Home è un **muro di locandine** reali, seguito da una lista di serate, ricerca e filtri; la mappa è una fascia dedicata e ha una modalità a tutto schermo. Tutto è navigabile anche senza JavaScript (le pagine arrivano già composte dal server, così i link sono condivisibili e le anteprime social funzionano).

## La Home

- In cima ci sono **fino a quattro locandine vere**, scelte tra le serate in evidenza o tra le prossime in programma. Sono immagini caricate dagli artisti o dai locali: nessuna foto di repertorio.
- Il titolo della Home è **"Stasera si suona."**: un invito diretto, non uno slogan pubblicitario.
- Sotto il muro c'è la **lista delle prossime serate**, con data grande, titolo, luogo, genere, orario e ingresso. È una lista editoriale, non una griglia di schede.
- Se cerchi o filtri, cambiano i risultati della lista; il muro resta la copertina della pagina.
- Le serate scelte dallo staff per la vetrina alimentano il muro. **Non si può pagare per comparire**: non esiste un algoritmo "pay-to-play".
- Sotto la lista c'è la sezione **Città**: mostra al massimo **sei città**, in **ordine alfabetico**, con il numero di serate future e la prossima data. Il link **Tutte le città** porta a `/citta`. Le città non sono un elenco fisso: entrano quando c'è almeno una serata pubblicata e vengono lette dal database.
- Dopo l'accesso, nella **navbar desktop** il pulsante **Dashboard** è evidenziato: non è più un link grigio confuso tra le altre voci.

## La mappa

- La mappa sta in una **fascia a tutta pagina** sotto la lista e si carica solo quando serve. Ha un **globo 3D** che ruota una volta sola, come introduzione, e si ferma al primo tocco. Se il tuo sistema chiede "meno animazioni", la rotazione non parte nemmeno.
- I **pin** sono gli eventi futuri. Più serate nello **stesso locale** diventano **un pin solo** con il numero delle date, per non avere tre puntini sovrapposti.
- Toccando un pin si apre una **scheda compatta**: data e ora nel fuso dell'evento, titolo, locale, genere, tipo e ingresso. Su desktop è un popup ancorato al pin; su mobile è un **bottom sheet** che sale dal basso.
- Se il locale ha più serate, le scorri **una alla volta** con le frecce e il contatore `‹ 1 / 3 ›`; la scheda resta della stessa altezza e non esce dalla mappa.
- Dalla scheda: **Pagina evento**, **Biglietti** (se ci sono), **Apri in Google Maps**, **Salva** e **Condividi**. I pulsanti hanno un'area di tocco di almeno 44 px.
- Il popup desktop lascia spazio alla ricerca e ai filtri: non finisce più sotto il pannello dei filtri quando la mappa è a tutto schermo.
- Gli eventi che dichiarano una posizione **approssimata** mostrano un'area invece di un punto, e la pagina evento lo spiega.
- **A tutto schermo** (voce "Mappa" nella barra, oppure il pulsante sulla mappa) la mappa diventa l'intera pagina: è la modalità migliore per esplorare.
- **Su mobile:** un dito **scorre la pagina** (la mappa non "ruba" il gesto), la mappa si muove solo nella modalità a tutto schermo o con **due dita**. Così non resti mai bloccato dentro la mappa.
- Il pulsante **Vicino a me** chiede al browser la posizione precisa e centra la mappa lì. Senza il tuo consenso la mappa usa una posizione approssimativa (città), ricavata dalla connessione: non serve alcun permesso.

## La ricerca

Sopra la lista c'è la **barra di ricerca**. I suggerimenti arrivano **dal database di OpenStage**, non da un servizio esterno, e sono raggruppati per tipo: **città, artisti, locali, eventi, generi**. Scrivendo "druso" trovi il locale; scrivendo "rock" trovi il

genere.

## I filtri rapidi

| Filtro | Cosa fa |
|---|---|
| **Stasera** | Gli eventi di oggi (la serata italiana, calcolata sul fuso `Europe/Rome`) |
| **Questo weekend** | Sabato e domenica della settimana corrente |
| **Questa settimana** | I prossimi sette giorni |
| **Prossimi 30 giorni** | Il mese che viene |
| **Distanza** | Raggio in km intorno a un centro: la tua posizione approssimata, o quella scelta |
| **Genere** | Solo i generi realmente presenti negli eventi pubblicati |
| **Tipo di evento** | Concerto, dj set, festival, open mic, acustico, altro |
| **Ingresso gratuito** | Solo le serate a ingresso libero |

Ogni filtro **scrive nell'indirizzo della pagina**: puoi copiare il link e chi lo apre vede esattamente la tua stessa lista. La lista e la mappa leggono gli stessi parametri, quindi non possono mai mostrarti cose diverse. Filtrando, la pagina **resta dove sei**: non torna in cima.

## Muro, lista e mappa

- **Muro di locandine**: le immagini reali delle serate in evidenza o delle prossime date. Su desktop e mobile cambia la composizione, non il contenuto.
- **Lista a righe**: ogni serata ha data, titolo, artista/locale, genere, orario e prezzo o ingresso gratuito. Le righe sono cliccabili e portano alla pagina evento.
- I risultati sono **paginati a 12** per pagina, e i filtri restano attivi cambiando pagina.
- **Mappa**: la stessa ricerca applicata ai pin; il pulsante "Apri a schermo intero" porta a `/mappa`, dove la mappa occupa tutta la pagina. La Home non usa più l'interruttore schede/tabella.
- **In evidenza**: le serate scelte a mano dallo staff compaiono nel muro. Non esiste una sezione pubblicitaria separata e **non si può pagare per comparire**.
- Se una ricerca non trova nulla, la pagina **non ti lascia a mani vuote**: ti spiega che il database è ancora piccolo e ti invita a segnalare locali e artisti che conosci.

## Salvare un evento

Il pulsante **Salva** scrive davvero nel database, sul tuo profilo (serve l'accesso). I preferiti sono raccolti in **Dashboard** → **Preferiti** (`/dashboard/saved`), dal più recente. Se non hai una dashboard da artista o locale, la voce è disponibile anche per i fan.

# Reclamare la tua scheda

Se una scheda artista o locale è stata creata dalla redazione, puoi diventarne proprietario.

## Artista

1. Apri la tua scheda in **Artisti**.
2. Premi **Sono io** e inserisci un **link pubblico** che provi che la scheda è tua (il tuo sito, una pagina social, una pagina con il tuo nome).
3. Il collegamento è **immediato**. Se gestisci già un'altra scheda artista, il sistema te lo dice: un profilo amministra una scheda sola.

## Locale

1. Apri la scheda del locale in **Locali** e premi **Sono il gestore**.
2. Inserisci la prova e un messaggio: la richiesta va in **approvazione** dallo staff.
3. Finché è in attesa la vedi in **Account**, e puoi **ritirarla**. Non puoi creare una seconda scheda mentre la richiesta è aperta.
4. Quando l'admin approva, la scheda diventa tua e ricevi il badge **Verificato**.

La verifica non è un badge decorativo: è il permesso che consente la pubblicazione diretta delle serate. Il primo evento di un account appena verificato può passare dalla revisione dello staff; dopo, le pubblicazioni sono automatiche.

# Pubblicare una serata

La dashboard è il posto dove gestisci il tuo profilo e le tue date.

| Voce | Cosa trovi |
|---|---|
| **Panoramica** | Bozze, pubblicate, annullate e preferiti |
| **I miei eventi** | Tutte le tue serate, divise tra in programma e **passate** |
| **Crea evento** | Il modulo di pubblicazione |
| **Preferiti** | Le serate che hai salvato |
| **Profilo / Account** | La tua scheda, la richiesta di reclamo e le impostazioni |

Nel modulo puoi inserire titolo, artista, data e ora, locale o nome del posto, indirizzo con pin sulla mappa, genere, tipo, prezzo o ingresso gratuito, link biglietti, locandina e il testo **"Come arrivare"** per i posti difficili da trovare. La posizione può essere **esatta o approssimata**: se scegli l'area, la pagina evento mostra una zona e non un punto finto.

Le serate nascono in **bozza**: puoi pubblicarle, annullarle o modificarle. Un locale può collegare la scheda di un artista esistente; se una serata simile è già presente, il sistema ti avvisa e ti lascia decidere. C'è un tetto anti-spam di **10 serate nelle 24 ore** per account, imposto dal database.

La **serata passata** non sparisce dalla tua dashboard: finisce nella sezione "Passate". Puoi eliminarla definitivamente (con conferma) se è un doppione o un errore; l'admin può farlo su qualsiasi scheda.

# Scrivere allo staff

La pagina **Contatti** (`/contatti`) ha un modulo vero: il messaggio viene salvato nel database, non affidato a un semplice link `mailto:`. Servono nome, email, un argomento (domanda generale, artista, locale, segnalazione, privacy o altro) e il testo; il modulo ha un campo nascosto anti-bot e limiti di lunghezza.

Dopo l'invio vedi una conferma. Lo staff legge i messaggi in **Admin** → **Messaggi** (`/admin/messaggi`), li filtra per **nuovi**, **letti** e **chiusi** e può rispondere via email. In questa fase **non parte un'email automatica**: la risposta usa il client di posta di chi gestisce la coda. La casella ufficiale e la risposta integrata arriveranno con la Fase 7.

# Segnalare una serata

Nella pagina di una serata in programma, chi ha un accesso può usare **Segnala questa serata** e scegliere un motivo (informazioni sbagliate, spam, contenuto offensivo, altro). La segnalazione arriva solo allo staff: **non è una recensione e non è pubblica**. Dopo tre segnalazioni da persone diverse e verificate, la serata va in pausa automaticamente e aspetta un controllo umano; l'admin può rimetterla online, annullarla o sospendere l'autore.

# Navigare il sito

| Voce | Cosa trovi |
|---|---|
| **Esplora** (Home) | Muro di locandine, lista delle prossime serate, ricerca, filtri e mappa |
| **Mappa** | La mappa a tutto schermo, con gli stessi filtri della Home |
| **Artisti** | Directory reale con ricerca, filtri, schede e concerti futuri |
| **Locali** | Directory reale con ricerca, filtri, schede e programmazione |
| **Per gli Artisti** | Pagina editoriale reale: cosa ottiene un artista, reclamo e pubblicazione |
| **Per le Venue** | Pagina editoriale reale: programmazione, dashboard e pubblico locale |
| **Chi siamo** | Pagina editoriale reale: principi, storia e cosa è (e non è) OpenStage |
| **Città** | Indice completo su `/citta`; la Home ne mostra solo sei, in ordine alfabetico |
| **Accedi / Registrati** | Accesso e creazione account |
| **Dashboard** (dopo l'accesso) | I miei eventi, crea evento, preferiti, profilo e impostazioni; il pulsante è evidenziato in navbar |
| **Admin** (solo staff) | Coda dei reclami, moderazione delle serate e coda messaggi |

> Le pagine di servizio (Privacy Policy, Termini, Cookie Policy, Chi siamo, Contatti, FAQ) sono raggiungibili dal **footer** in fondo a ogni pagina. Chi siamo, Contatti e FAQ sono reali; Privacy, Termini e Cookie sono ancora segnaposto e lo dicono apertamente.

## Lingua

La piattaforma parte in **italiano** se ti colleghi dall'Italia, in **inglese** da qualsiasi altro paese. Puoi cambiarla quando vuoi con il selettore **IT / EN** nella barra in alto: la lingua vive nell'indirizzo (`/it/...` oppure `/en/...`), quindi un link mantiene la lingua anche quando lo condividi. La scelta viene ricordata alla visita successiva.

## Il player in basso (non ancora attivo)

È previsto un **player Spotify fisso** in fondo allo schermo, per ascoltare in background le playlist di OpenStage mentre navighi. **Oggi però non si vede**: il player esiste nel codice ma compare **solo** quando è configurata una playlist pubblica, e le playlist di OpenStage non sono ancora state create. Non c'è niente di rotto: è una funzione **spenta**, non una funzione mancante. Quando le playlist esisteranno, il player si accende con una sola impostazione, senza modifiche al codice.

# Domande frequenti

**Posso avere due profili (uno da fan e uno da artista) con la stessa email?** No: ogni email è un account. Se hai scelto artista e volevi fare il fan, puoi comunque esplorare tutto e salvare i preferiti: le funzioni da fan restano disponibili.

**Ho scelto Locale ma sono un musicista: posso cambiare?** No, il ruolo è definitivo. Scrivi allo staff: solo l'admin può correggere.

**Perché mi ha chiesto il ruolo dopo la conferma email o dopo Google?** Il ruolo si sceglie **dopo** aver verificato l'email, così un errore di battitura non crea un profilo artista o locale sbagliato. Con Google non arriva nessuna email di conferma perché Google ha già verificato l'indirizzo; la schermata del ruolo è quindi il primo passaggio.

**Devo pagare qualcosa?** No. Il piano base è gratuito per fan, artisti e locali: è un principio del progetto.

**Perché sulla mappa vedo un pin con più date?** Perché quel locale ospita più serate: raggrupparle in un punto solo evita pin sovrapposti e rendere illeggibile la mappa. Le date si scorrono con le frecce nella scheda.

**Perché su mobile la mappa non si muove quando trascino?** È voluto: un dito serve a **scorrere la pagina**, altrimenti resteresti intrappolato dentro la mappa. Per muoverla usa **due dita**, oppure aprila a tutto schermo.

**Perché un evento dice "non ancora rivendicato"?** Perché né l'artista né il locale hanno ancora reclamato la loro scheda. Puoi reclamarla dalla scheda stessa: per un artista il collegamento è immediato con un link di prova; per un locale serve l'approvazione dello staff.

**Ho salvato un evento: dove lo ritrovo?** In **Dashboard** → **Preferiti** (`/dashboard/saved`), dal più recente. Il salvataggio è reale e resta legato al tuo account.

**Come pubblico una serata?** Dalla **Dashboard** → **Crea evento**, se il tuo profilo è artista o locale. Se sei ancora un fan, prima devi scegliere il ruolo; la scelta è definitiva. Il primo evento può passare dalla revisione dello staff, poi la pubblicazione è automatica.

**Non trovo eventi nella mia città: la piattaforma è rotta?** No. Il database cresce con i locali e gli artisti che pubblicano: all'inizio ci sono poche città. Se conosci un locale o un artista che dovrebbe esserci, segnalalo.

**Dove trovo tutte le città?** Nell'indice `/citta`, raggiungibile dal link "Tutte le città" nella sezione Città della Home e dal footer. La Home mostra solo sei città in ordine alfabetico, così non diventa un elenco lunghissimo.

**Perché la creazione evento non compare?** Perché il tuo profilo è ancora **fan**. La creazione è riservata ad artisti e locali: scegli il ruolo dalla barra in alto (la scelta è definitiva) oppure reclama la tua scheda esistente.

# Architettura e stack

OpenStage è una **applicazione web Next.js** (App Router, TypeScript) renderizzata **lato server** su tutte le pagine pubbliche: ogni pagina arriva già composta, così SEO e anteprime social funzionano sempre e non esiste una "SPA" vuota da riempire nel browser.

| Pezzo | Tecnologia | Versione | Perché |
|---|---|---|---|
| Framework | Next.js (App Router) | 15.5.28 | SSR ovunque, server actions, ottimo su Vercel |
| UI | React | 19.1.0 | Richiesto da Next 15 |
| Linguaggio | TypeScript | 5.x | Errori presi prima del deploy |
| Stile | Tailwind CSS | v4 | Utility-first, tema custom con i token del brand |
| Animazioni | CSS + `IntersectionObserver` | — | Reveal, stagger e hover leggeri. **Framer Motion** è stato valutato in Fase 5 e rimandato alla Fase 13: per un fade aggiungeva ~45 KB gzip |
| Database + Auth | Supabase (Postgres) | eu-north-1 | RLS nativa sull'`auth.uid()`; un solo fornitore per dati e login |
| Anti-bot | Cloudflare Turnstile + Supabase Auth | — | Captcha su registrazione, login e reset; la verifica `siteverify` è lato Supabase |
| Mappe | MapLibre GL JS + **OpenFreeMap** | 6.14.0 | Mappa e globo degli eventi: **nessuna chiave API, nessun account** (Fase 2) |
| Email | Resend + Aruba | — | Invio transazionale `eu-west-1`; ricezione sulle caselle Aruba (`noreply@`, `ciao@`) |
| Pagamenti | PayPal | *in arrivo* | Solo funzioni opzionali: il piano base resta gratuito |
| Hosting | Vercel | Hobby | Deploy continuo da GitHub |
| Repository | GitHub | `openstagemusic/openstage` | Ramo principale `main` |

## Alternative valutate e perché sono state scartate

| Alternativa | Motivo del no |
|---|---|
| Neon + Clerk | Due fornitori da configurare, e Clerk non ha RLS sul database: la sicurezza dei dati va riscritta a mano nell'app |
| Nhost / Appwrite / Firebase | Livello database meno "Postgres puro" (o NoSQL): vincoli, viste e policy SQL perdono potenza |
| Turso | SQLite distribuito: ottimo per altri casi, ma qui serve Postgres con RLS nativa |
| Supabase | **Scelto**: Postgres con Row Level Security collegata all'utente loggato, Auth integrata, piano gratuito ampio |
| MapTiler | **Scartato in Fase 2**: richiede account e chiave API. OpenFreeMap non chiede né l'uno né l'altro, consente l'uso commerciale e si "paga" con l'attribuzione obbligatoria |

| Alternativa | Motivo del no |
|---|---|
| Globe.gl | **Scartato in Fase 2**: la proiezione "globo" è nativa in MapLibre, quindi una dipendenza in meno da mantenere |

## Versioni dell'ambiente

- **Node.js**: 24.x (in locale 24.16.0, su Vercel `nodeVersion` 24.x)
- **npm**: 11.x
- **Regione Supabase**: `eu-north-1` (dati in Europa)
- **Progetto Supabase (ref)**: `tzdigzvkneonkhkwoeed`
- **Mappa**: MapLibre GL JS 6.14.0 con lo stile `https://tiles.openfreemap.org/styles/dark`

# Requisiti, comandi e struttura del progetto

| Comando | Cosa fa |
|---|---|
| `npm install` | Installa le dipendenze |
| `npm run dev` | Avvia lo sviluppo locale su `http://localhost:3000` (Turbopack) |
| `npm run build` | **Build di produzione**: va eseguita a ogni fine fase, senza errori |
| `npm run start` | Avvia la build già compilata |
| `npm run lint` | Controlla il codice con ESLint |
| `npm run manuale` | Rigenera **questo PDF** dai sorgenti in `docs/manuale/` |

Per allineare le variabili d'ambiente locali a quelle di Vercel: `npx vercel env pull .env.local`.

## Struttura delle cartelle

```
app/                        Pagine (App Router) e server action
  layout.tsx                Navbar, footer, player, barra mobile, lingua
  page.tsx                  Home (Esplora): muro di locandine, lista, ricerca, filtri, mappa
  mappa/                    Mappa a tutto schermo (stessi filtri della Home)
  eventi/[...percorso]      Pagina evento SSR con dati strutturati e anteprima social
  artisti/ locali/          Directory reali con ricerca, filtri e schede [slug]
  dashboard/                Panoramica, eventi, editor, preferiti, account
  admin/                    Coda reclami, moderazione serate e coda messaggi (solo staff)
  admin/messaggi/           Messaggi dal modulo contatti, con filtri e stati
  account/                  Profilo, richiesta di reclamo, impostazioni
  onboarding/role/          Scelta del ruolo ("Diventa artista o locale")
  onboarding/artist/        Profilo e reclamo artista
  onboarding/venue/         Profilo e reclamo locale
  citta/                    Indice città e pagina /citta/[city], generate dai dati reali
  per-gli-artisti/ per-le-venue/ about/ contatti/ faq/ privacy/ terms/ cookies/
  login/ register/          Accesso e registrazione
  forgot-password/          Richiesta del link di recupero
  reset-password/           Nuova password dopo il link di recovery
  auth/callback/            Ritorno da Google: crea/aggiorna la sessione
  api/suggest/              Endpoint dei suggerimenti di ricerca
  api/geocode/              Geocodifica e controllo pin/città (Photon)
  actions/locale.ts         Server action che salva la lingua scelta
  actions/contact.ts        Server action del modulo contatti (scrive in database)
  actions/messages.ts       Cambio stato di un messaggio (solo admin)
components/
  layout/                   Navbar, Footer, MobileTabBar, SpotifyPlayerDock
  auth/                     Form di accesso/registrazione, RolePicker, Turnstile
  discovery/                SearchBar, QuickFilters, ViewSwitcher, useScrollAnchor
  map/                      GlobeMap, MapCanvas, EventCard, VenueEventsCard
  events/                   EventTeaser, EventPageView, EventCountdown, ShareEventPanel,
                            AddToCalendar, ReportEventButton, SaveButton
  home/                     HeroPosters, EventRows, CityStrip, HomeIntro, BentoGrid, Counters
  directory/                ArtistCard, VenueCard, DirectoryFilters, VerifiedBadge
  dashboard/                DashboardShell, MyEvents, EventEditor, LocationPicker,
                            CoverField, EventPreviewCard
  claims/                   ClaimPanel, PageSearch, PendingRequestPanel
  admin/                    AdminQueue, AdminEventsPanel, AdminMessagesPanel
  motion/                   Reveal (IntersectionObserver)
  ui/                       Pagination, PagePlaceholder
  i18n/                     LanguageSwitcher
i18n/dictionaries/          it.ts e en.ts: tutti i testi dell'interfaccia
lib/
  auth.ts                   isAdminEmail() per le decisioni di interfaccia
  auth/roles.ts             Tipo UserRole, etichette, permessi
  cities/                   Query dell'indice città e percorsi pubblici
  claims/                   Query e azioni del reclamo
  dashboard/                Query della dashboard e dei preferiti
  directory/                Query di artisti e locali
  messages/                 Query della coda messaggi e cambio stato
  events/                   filters, queries, types, format, timezone, paths,
                            calendar, saved, cover
  geo/                      città normalizzata, geocodifica, controllo del pin
  map/                      config (URL dello stile), venues (raggruppamento dei pin)
  moderation/               Coda e azioni dell'admin sulle serate
  covers/                   Disegno canvas di locandine e Share Card
  seo/                      canonical, JSON-LD, card Open Graph
  pagination.ts             ITEMS_PER_PAGE: costante condivisa (vedi "errori noti")
  i18n/                     config, server, metadata, dictionaries
  supabase/                 client, server, admin, env, middleware, session
supabase/migrations/        0001…0028 (vedi tabella migrazioni)
docs/                       Piani di fase, review di fase, sorgenti del manuale
scripts/manuale.mjs         Generatore del PDF (solo locale, non va in produzione)
```

`middleware.ts` (nella radice) rinnova la sessione Supabase a ogni richiesta e, alla prima visita, sceglie la lingua in base al paese rilevato da Vercel.

# Variabili d'ambiente

Le stesse variabili esistono in **tre ambienti** su Vercel (Production, Preview, Development): quando ne aggiungi una, impostala su tutti e tre.

| Variabile | Dove si prende | A cosa serve |
|---|---|---|
| `NEXT_PUBLIC_SUPABASE_URL` | Supabase → Project Settings → API | Indirizzo del progetto Supabase |
| `NEXT_PUBLIC_SUPABASE_ANON_KEY` | Supabase → Project Settings → API | Chiave pubblica (protetta dalla RLS) |
| `SUPABASE_SERVICE_ROLE_KEY` | Supabase → Project Settings → API | Chiave di servizio: **solo server**, usata dal client admin per operazioni di manutenzione. Non va mai esposta al browser |
| `NEXT_PUBLIC_TURNSTILE_SITE_KEY` | Cloudflare → Turnstile | Chiave pubblica del captcha |
| `TURNSTILE_SECRET_KEY` | Cloudflare → Turnstile | Verifica lato server (Supabase la usa per il captcha) |
| `RESEND_API_KEY` | Resend → API Keys (`openstage-vercel`) | Invio email transazionali dall'app |
| `RESEND_SMTP_URL` | Resend → Domains → SMTP | Configurazione SMTP di Supabase Auth (host `smtp.resend.com`, porta 587, utente `resend`) |
| `EMAIL_FROM` | Scelta interna | Mittente, es. `OpenStage <noreply@getopenstage.it>` |
| `EMAIL_REPLY_TO` | Scelta interna | Risposte, es. `ciao@getopenstage.it` |
| `ADMIN_EMAIL` | Scelta interna | Email dello staff: deve combaciare con `settings.admin_email` |
| `NEXT_PUBLIC_SITE_URL` | Dominio di produzione | URL canonico per SEO e link di ritorno |
| `NEXT_PUBLIC_SPOTIFY_PLAYLIST_ID` | URL della playlist (la parte dopo `/playlist/`) | Player Spotify in fondo alle pagine. **Oggi è vuota**: senza un id il player non compare affatto (nessun contenuto finto, nessun player vuoto) |
| `GEOCODER_TIMEOUT_MS` | Facoltativa, solo per le prove locali | Tempo massimo di attesa di Photon. In produzione il valore predefinito è 5 s; in locale si può alzare per non far fallire il controllo del pin |

> `.env.local` **non deve mai** finire su Git: il file è già escluso. Il modello da copiare è `.env.example`.
> Due variabili sono marcate **sensitive** su Vercel (`SUPABASE_SERVICE_ROLE_KEY`, `TURNSTILE_SECRET_KEY`): una volta salvate non sono più leggibili, per questo è normale non poterle "rivedere" dalla dashboard.

**Le Fasi 2, 3, 4 e 5.4 non hanno aggiunto variabili obbligatorie**: la mappa usa OpenFreeMap e la geocodifica usa Photon, che non richiedono chiavi API né account. Vale la regola generale: quando una fase introduce un servizio nuovo, la variabile va impostata su **tutti e tre** gli ambienti Vercel (Production, Preview, Development) prima del test in produzione.

# Autenticazione e gestione del ruolo

## Come funziona l'accesso

```
Email + password:  form → Supabase Auth (CAPTCHA) → email di conferma → /auth/callback
                                  → trigger handle_new_user() crea la riga in profiles
                                  → se manca il ruolo, si passa da /onboarding/role
                                  → il ruolo viene salvato in profiles e user_metadata

Google:            bottone → Google → /auth/callback → sessione
                                  → se manca il ruolo, si passa da /onboarding/role

Reset password:    /forgot-password → Supabase (CAPTCHA) → email Resend
                                  → link → /reset-password → updateUser({ password })
```

Punti importanti:

1. **Il database è la fonte di verità del ruolo.** Il ruolo vive in `public.profiles.role`; il badge in navbar e le pagine protette leggono da lì (helper `getSessionProfile()`, con `cache()` di React: una sola lettura per richiesta, condivisa tra layout e pagina).
2. **Google non trasporta il ruolo.** Non lo si passa nell'URL di redirect perché sarebbe un parametro che il browser può manipolare. Perciò, al primo ingresso con Google, il ruolo si sceglie in una pagina dedicata e viene salvato sia nel profilo sia nei metadati (`user_metadata.role`), che servono come marcatore.

## Dove sono applicate le regole del ruolo

Le regole valgono su **tre livelli**, dal più importante al più visibile:

| Livello | Cosa fa | Dove |
|---|---|---|
| **Database** | Rifiuta le transizioni di ruolo non ammesse e le righe incoerenti | Trigger `prevent_role_escalation` + policy RLS |
| **Pagine** | Un fan non entra nelle pagine da artista, e viene indirizzato alla scelta del ruolo | `app/onboarding/artist`, `app/events/new`, `app/onboarding/role` |
| **Interfaccia** | Mostra il ruolo e le azioni giuste (badge + link contestuale) | `components/layout/Navbar.tsx`, `lib/auth/roles.ts` |

Regola delle transizioni, in una riga: **solo** `fan → artista | locale`, una volta sola. Tutto il resto viene annullato. L'admin può correggere (eccezione voluta, e anche le modifiche fatte direttamente in SQL dalla dashboard non vengono bloccate).

# Database

## Tabelle

`profiles` — una riga per utente (creata dal trigger alla registrazione). `id` (chiave, uguale a `auth.users.id`), `email`, `full_name`, `role` (**enum**: `fan` / `artist` / `venue` / `admin`, predefinito `fan`), `city`, `verified_at` (verifica dello staff), `events_daily_limit` (tetto anti-spam, vuoto = 10), `suspended_at`, `created_at`, `updated_at`.

`artists` — scheda pubblica dell'artista. `id`, `profile_id` (unico, **nullable**: un artista può esistere senza account), `name`, `slug` (unico, per l'URL pubblico), `city`, `city_key` (generata, per i filtri), `genre`, `spotify_url`, `bio`, `created_at`, `updated_at`.

`venues` — scheda pubblica del locale/organizzatore. `id`, `profile_id` (unico, nullable), `name`, `slug`, `city`, `city_key`, `address`, `latitude`, `longitude`, `capacity`, `website`, `created_at`, `updated_at`.

`events` — gli eventi. `id`, `slug`, `title`, `description`, `artist_id`, `venue_id`, `venue_name` (locale senza scheda), `starts_at`, `city`, `city_key`, `status` (**enum**: `draft` / `published` / `cancelled`), `created_by`, `created_at`, `updated_at`, più i campi della scoperta: `latitude`, `longitude`, `location_precision` (**enum**: `exact` / `approximate`), `area_radius_m`, `directions_text` ("Come arrivare"), `genre`, `kind` (**enum**: `concert` / `dj_set` / `festival` / `open_mic` / `acoustic` / `other`), `is_free`, `price_text`, `ticket_url`, `cover_image_url`, `is_featured`, `featured_until`, `timezone` (predefinito `Europe/Rome`). Per la moderazione: `cancel_reason`, `merged_into`, `merged_at`, `merged_status`, `auto_hidden_at`, `auto_hidden_status`. `is_unclaimed` **non esiste più**: lo stato "inserito da OpenStage" si calcola dall'assenza dei `profile_id` collegati.

`saved_events` — gli eventi salvati da un profilo (Fase 2). `profile_id`, `event_id`, `created_at`, con chiave primaria composta. È un segnalibro: nessun dato sensibile, e la RLS lo limita al proprietario.

`claims` — richieste di reclamo di una scheda (Fase 3). `subject_type` (`artist` / `venue`), `artist_id` / `venue_id`, `profile_id`, `status` (`pending` / `approved` / `rejected` / `revoked` / `withdrawn`), `proof_url` obbligatorio, `note`, date e chi ha deciso. Una sola richiesta pendente per persona e scheda.

`reports` — segnalazioni delle serate (Fase 4). Una per persona per evento, motivi chiusi, nota facoltativa, stato `open` / `resolved` / `dismissed`. Non è una recensione: la legge solo l'admin.

`event_creations` — registro append-only delle serate create, usato dal tetto anti-spam (10 nelle 24 ore). Non si cancella insieme all'evento, altrimenti la quota si aggirerebbe eliminando la bozza.

`settings` — parametri interni chiave/valore. Oggi contiene `admin_email` = email dello staff. È leggibile solo dall'admin.

## Funzioni e trigger

| Oggetto | Cosa fa |
|---|---|
| `handle_new_user()` | Alla registrazione crea la riga in `profiles` e assegna il ruolo (o `admin` se l'email è quella dello staff) |
| `prevent_role_escalation()` | Blocca le transizioni di ruolo non ammesse (vedi sopra) |
| `touch_updated_at()` | Aggiorna `updated_at` a ogni modifica |
| `is_admin()` | Vero se l'utente loggato è l'email in `settings.admin_email`. Usata dalle policy |

| Oggetto | Cosa fa |
|---|---|
| `search_events(...)` | Ricerca e filtri di Home e mappa: testo, preset temporali, genere, tipo, ingresso gratuito, distanza e paginazione in **una sola query**. `SECURITY INVOKER`, con `status = 'published'` esplicito. Dalla Fase 4 la distanza si misura dal **bordo** di una zona approssimata |
| `search_suggestions(...)` | I suggerimenti della barra di ricerca, raggruppati per tipo: città, artisti, locali, eventi, generi |
| `claim_artist(...)`, `request_venue_claim(...)`, `decide_claim(...)`, `withdraw_claim(...)` | Reclamo: artista immediato con prova, locale in approvazione, ritiro della richiesta e decisione dell'admin |
| `merge_event(...)`, `unmerge_event(...)` | Unione reversibile di due schede doppione; sposta anche i Preferiti. Solo admin |
| `city_key(...)` | Minuscolo, senza accenti e senza provincia: "Milano" e "milano (MI)" sono la stessa città |
| `event_daily_limit(...)` | Tetto anti-spam per account, predefinito 10 serate nelle 24 ore |
| `reports_apply_threshold()` | Alla terza segnalazione da persone diverse e verificate mette la serata in pausa (bozza) |
| Trigger | `on_auth_user_created` (su `auth.users`), `profiles_guard`, `artists_touch`, `venues_touch`, `events_touch`, `settings_touch`, `events_rate_limit`, `events_log_creation`, `reports_apply_threshold` |

## Indici

Chiavi primarie e unicità su `profiles.id`, `artists.profile_id`, `venues.profile_id`, `settings.key`, più gli indici per le ricerche frequenti:

| Indice | A cosa serve |
|---|---|
| `events_starts_at_idx`, `events_published_starts_at_idx` | Ordinare e filtrare per data (il secondo solo sugli eventi pubblicati) |
| `events_city_idx`, `events_published_geo_idx` | Filtro per città e per posizione (latitudine/longitudine) |
| `events_artist_id_idx`, `events_venue_id_idx`, `events_created_by_idx` | I collegamenti tra tabelle |
| `events_featured_idx` | La sezione "In evidenza" |
| `events_title_trgm_idx`, `events_city_trgm_idx`, `artists_name_trgm_idx`, `venues_name_trgm_idx` | Ricerca testuale "contiene" con l'estensione `pg_trgm`: è ciò che rende veloci i suggerimenti |
| `saved_events_event_idx` | Gli eventi salvati, letti per evento |

> Gli strumenti di Supabase segnalano alcuni di questi indici come "mai usati". Con poche righe il pianificatore preferisce leggere tutta la tabella, quindi è un artefatto del volume di dati attuale, non un errore di progetto: da rimisurare con `EXPLAIN` quando gli eventi saranno centinaia (backlog B04).

## Migrazioni

Le migrazioni stanno in `supabase/migrations/` e vanno applicate **in ordine**, una volta sola:

| File | Contenuto |
|---|---|
| `0001_init.sql` | Schema iniziale: tabelle, enum, policy, trigger, impostazioni |
| `0002_role_self_service.sql` | Il ruolo `fan` / `artista` / `locale` è autodichiarabile (l'admin no) |
| `0003_hardening.sql` | Indurimento: `search_path`, revoche, indici, policy riscritte con `(select auth.uid())` |
| `0004_role_lock.sql` | Blocco delle transizioni di ruolo + coerenza del ruolo nella RLS |
| `0005_events_discovery.sql` | Campi della scoperta su `events`: coordinate, posizione approssimata, genere, tipo, ingresso gratuito, biglietti, vetrina, fuso orario |
| `0006_saved_events.sql` | Tabella `saved_events` (eventi salvati) con la sua RLS |
| `0007_artists_unclaimed.sql` | `artists.profile_id` diventa **nullable**: un artista può esistere senza account, come i locali |
| `0008_search_rpc.sql` | Funzioni `search_events` e `search_suggestions`: ricerca, filtri, distanza e paginazione nel database |
| `0009_slugs.sql` | Slug di artisti, locali ed eventi, capienza dei locali, verifica del profilo, tabella `claims` |
| `0010_search_slugs.sql` | Ricerca e suggerimenti con gli slug |
| `0011_claims.sql` | Funzioni di reclamo e pubblicazione riservata agli account verificati |
| `0012_claim_ownership.sql` | Proprietà della scheda reclamata e sostituzione della scheda vuota |
| `0013_event_unclaimed_derived.sql` | "Non reclamato" diventa un valore calcolato; la colonna salvata viene eliminata |
| `0014_claim_pending.sql` | Una richiesta in attesa alla volta, blocco della creazione, ritiro |
| `0015_event_fields.sql` | `price_text` e `venue_name` per l'editor |
| `0016_search_events_venue_price.sql` | Ricerca con locale e prezzo |
| `0017_geocode_cache.sql` | Cache della geocodifica |
| `0018_admin_publish_event.sql` | Pubblicazione per conto terzi dall'admin |
| `0019_event_covers.sql` | Bucket e policy per le locandine |
| `0020_event_anti_spam.sql` | Tetto di 10 serate nelle 24 ore e sospensione dell'account |
| `0021_profiles_field_guard.sql` | Campi del profilo protetti da modifiche non ammesse |
| `0022_event_cancellation.sql` | Annullamento delle serate con motivo |
| `0023_event_save_hygiene.sql` | Igiene del salvataggio e "Come arrivare" |
| `0024_city_quality.sql` | `city_key`, raggio onesto, città generata |
| `0025_event_merge.sql` | Unione reversibile dei doppioni |
| `0026_reports.sql` | Segnalazioni, auto-pausa e pannello di moderazione |
| `0027_merge_admin_check.sql` | `merge_event` usa `is_admin()`, una sola definizione di admin |

| File | Contenuto |
| --- | --- |
| `0028_contact_messages.sql` | Tabella, indici e RLS dei messaggi dal modulo contatti |

> Lo storico ufficiale è nella tabella `supabase_migrations.schema_migrations` (voci `0001` … `0028`). Se si aggiunge una migrazione, va aggiunta **anche** quella riga, altrimenti `supabase migration list` non la vede.

# Sicurezza: Row Level Security

Tutte le tabelle hanno la **RLS attiva**: senza una policy che lo consenta, una riga non è visibile né modificabile. Regola d'oro: le policy parlano di `auth.uid()` (l'utente loggato) e di `is_admin()`, mai di valori passati dal browser.

| Tabella | Policy | Regola in parole semplici |
|---|---|---|
| `profiles` | `profiles_select_own_or_admin` | Vedi solo il tuo profilo (l'admin tutti) |
| | `profiles_update_own_or_admin` | Modifichi solo il tuo (l'admin tutti) — il ruolo passa dal trigger |
| `artists` | `artists_select_public` | Le schede artista sono pubbliche |
| | `artists_insert_own` | Crei la tua riga **solo se il tuo ruolo è** `artist` |
| | `artists_update_own`, `artists_delete_own` | Gestisci solo la tua riga (o l'admin) |
| `venues` | `venues_select_public` | Le schede locale sono pubbliche |
| | `venues_insert_own` | Crei la tua riga **solo se il tuo ruolo è** `venue` |
| | `venues_update_own`, `venues_delete_own` | Gestisci solo la tua riga (o l'admin) |
| `events` | `events_select_public` | Vedi gli eventi **pubblicati**; i tuoi li vedi anche in bozza, l'admin li vede tutti |
| | `events_insert_authenticated` | Crei un evento **solo se sei** `artist` **o** `venue` e come tuo |
| | `events_update_own`, `events_delete_own` | Modifichi solo i tuoi eventi (o l'admin) |
| `saved_events` | `saved_events_select_own` | Vedi solo i **tuoi** preferiti |
| | `saved_events_insert_own`, `saved_events_delete_own` | Salvi e togli solo i tuoi |
| `claims` | `claims_select_own_or_admin` | Vedi la tua richiesta (o tutte se admin); si scrive solo tramite funzioni |
| `reports` | `reports_insert_own` | Segnali solo a nome tuo, una volta per serata |
| | `reports_select_own_or_admin`, `reports_admin_update` | Vedi le tue segnalazioni; l'admin le legge e le chiude |
| `event_creations` | `event_creations_read_admin` | Il registro anti-spam è leggibile solo dall'admin |
| `settings` | `settings_select_admin`, `settings_update_admin` | Solo l'admin |
| `contact_messages` | `contact_messages_insert_public` | Chiunque può inviare un messaggio, solo con stato `new` |
| | `contact_messages_select_admin`, `..._update_admin`, `..._delete_admin` | Legge, aggiorna e cancella solo l'admin |

**Perché le policy contano davvero:** un utente malintenzionato può chiamare l'API di Supabase direttamente, senza passare dall'interfaccia. Le regole "il fan non vede il pulsante" valgono solo per l'interfaccia; quelle che contano sono nel

database. Per esempio, un fan che prova a creare una riga artista via API riceve un **errore 42501 (row-level security policy)**, anche se la pagina web non gliela mostrerebbe mai.

## Avvisi di sicurezza attualmente accettati

Gli strumenti di Supabase segnalano ogni tanto dei punti da valutare. Oggi restano aperti, con motivazione:

| Avviso | Perché è accettato |
|---|---|
| `is_admin()` è `SECURITY DEFINER` ed eseguibile da anonymous/authenticated | Le policy RLS la invocano: deve poter leggere `settings` (che è riservata all'admin) senza esporre nient'altro |
| *Leaked password protection* disattivata | **Non è disponibile sul piano gratuito**: è una funzione del piano Pro. Va attivata quando si passerà al piano a pagamento per i pagamenti (backlog B01) |
| Indici "mai usati" | Con poche righe il pianificatore preferisce leggere tutta la tabella: è un artefatto del volume attuale, non un difetto di progetto (backlog B04) |
| `npm audit`: segnalazioni su dipendenze di sviluppo e build | Arrivano da `eslint-config-next` e da `next` → `postcss`: servono salti di versione maggiore, da pianificare (backlog B03) |

# Come si collauda la RLS

Non serve un browser: si impersona un utente direttamente in SQL e si legge il risultato. Con gli strumenti MCP di Supabase funziona così:

```
-- 1. Ci si mette nei panni di un utente loggato…
set local role authenticated;
set local request.jwt.claims = '{"sub":"<uuid-utente>","email":"<email>","role":"authenticated"}';

-- 2. …si prova l'operazione. Se la RLS non la consente, arriva un errore 42501.
update public.profiles set role = 'venue' where id = '<uuid-utente>';

-- 3. Si torna a leggere come admin per verificare l'effetto reale.
reset role;
select role from public.profiles where id = '<uuid-utente>';
```

Due accortezze imparate sul campo:

1. **I claim restano per tutta la transazione.** Se nella stessa sessione provi dopo una modifica "da dashboard", la GUC dei claim è ancora attiva e il test mente. Azzerala con `set local request.jwt.claims = ''` prima di simulare l'SQL diretto.
2. Per catturare l'esito di un'operazione che deve **fallire**, avvolgila in un blocco `do $$ … exception when others then … $$` e salva l'esito con `set_config(..., true)`, così l'errore non interrompe tutto il resto del collaudo.

Vale la pena farlo per: transizioni di ruolo, insert in `artists` / `venues` / `events` con ruolo sbagliato, visibilità dei profili altrui.

# Internazionalizzazione

- Due dizionari: `i18n/dictionaries/it.ts` e `i18n/dictionaries/en.ts`.
- `en.ts` **è la fonte dei tipi**: se aggiungi una chiave solo in `it.ts`, la build fallisce. Aggiungere un testo significa quindi **sempre** toccare entrambi i file, con lo stesso nome.
- Lingua iniziale: l'italiano se il visitatore arriva dall'Italia ( `x-vercel-ip-country` ), altrimenti l'inglese. La scelta manuale (switcher IT/EN) viene salvata nel cookie `NEXT_LOCALE` e da quel momento ha la precedenza.
- Le pagine di supporto (Privacy, Termini, Cookie, Chi siamo, Contatti) sono bilingue come tutto il resto.

# Design system

| Elemento | Valore |
|---|---|
| Sfondo | `#07070D` |
| Viola (primario) | `#8B5CF6` |
| Ciano (secondario) | `#22D3EE` |
| Rosa (accento) | `#F472B6` |
| Titoli-poster (Home) | Anton |
| Titoli delle pagine | Space Grotesk |
| Etichette e metadati | IBM Plex Mono |
| Corpo del testo | Inter |
| Effetti | Glass solo su navbar, pop-up e pannelli; nella Home niente card, niente contenitori arrotondati |

Regole da non tradire:

1. **Glass solo su navbar, pop-up e pannelli**: mai dietro a testi lunghi, altrimenti la leggibilità crolla. La Home 5.4 non usa card: locandine, righe e bordi netti.
2. **Mobile first**: si progetta prima lo schermo piccolo.
3. **Movimento sobrio**: CSS + `IntersectionObserver`, solo `transform` / `opacity`, `prefers-reduced-motion` rispettato. Niente marquee infinito, niente parallax.
4. I token del tema stanno in `app/globals.css`: cambiando lì i colori, cambiano ovunque. I dettagli stanno in `docs/design-system.md`.

# Mappe e tile

La mappa è **MapLibre GL JS** con lo stile di **OpenFreeMap**. La scelta è stata fatta in Fase 2 al posto di MapTiler:

| Aspetto | MapTiler | OpenFreeMap (**scelto**) |
|---|---|---|
| Chiave API | Obbligatoria | **Nessuna** |
| Account | Obbligatorio | **Nessuno** |
| Uso commerciale | Solo nei piani a pagamento | **Consentito** |
| Costo | A consumo oltre la soglia gratuita | Gratuito |
| Attribuzione | Richiesta | Richiesta (già presente sulla mappa) |
| Garanzie | Contratto e SLA | **Nessuna**: è un'istanza finanziata da donazioni |

Il rischio accettato è l'ultima riga: se OpenFreeMap dovesse chiudere, la mappa resterebbe senza tile. La mitigazione è che **l'indirizzo dello stile è in un punto solo**, `lib/map/config.ts`: cambiare fornitore significa cambiare una riga.

Punti tecnici da ricordare:

1. **Il worker di MapLibre è servito dal nostro dominio** (`public/maplibre-gl-worker.mjs`, copiato da `scripts/copy-maplibre-worker.mjs` durante `prebuild`): così non dipendiamo da una CDN esterna e non serve allentare la Content Security Policy.
2. `maplibre-gl` **esporta solo nomi** (nessun export predefinito): si importa come `import { Map, Marker, Popup } from "maplibre-gl"`.
3. **Il pop-up deve restare dentro il riquadro della mappa.** Due accortezze, imparate sul campo: un locale con molte serate mostra **una serata alla volta** con le frecce numerate (altezza costante), e il margine della mappa va **dal lato giusto** (in alto per il pop-up desktop, che sta sopra il pin; in basso per il pannello mobile, che sta sotto). Senza queste due cose la scheda esce dalla mappa e viene tagliata. Nella 5.5B il margine superiore del popup è stato portato a **340 px** quando c'è il pannello ricerca/filtri, per non finire sotto l'overlay della pagina `/mappa`.
4. **Il globo è una proiezione nativa** di MapLibre: nessuna libreria aggiuntiva (Globe.gl è stato scartato proprio per questo).
5. **Su mobile le gesture sono "cooperative"** (`cooperativeGestures: true`): un dito scorre la pagina, la mappa si muove solo con due dita o nella modalità a tutto schermo.

# Ricerca, filtri e stato nell'URL

Tutto lo stato della scoperta — testo cercato, filtri, raggio e pagina — vive **nell'indirizzo della pagina**, non in memoria. (La vecchia vista schede/tabella non è più esposta in Home: la lista 5.4 è a righe.)

- un link filtrato è condivisibile e si apre già filtrato;
- la pagina si compone lato server, quindi funziona anche senza JavaScript;
- lista e mappa leggono **gli stessi parametri**: non possono mostrare cose diverse.

Scelte da non perdere:

| Scelta | Perché |
|---|---|
| Ricerca e filtri in **una funzione SQL** (`search_events`) e non nell'app | Ricerca testuale su più tabelle, paginazione corretta **con la distanza**, una sola fonte di verità |
| `status = 'published'` **esplicito** dentro la funzione | La funzione è `SECURITY INVOKER`: si comporta come l'utente che la chiama, e la regola "solo pubblicati" resta scritta a chiare lettere |
| Carattere di escape `!` nei confronti `LIKE` | In SQL non si usano i backslash: `!` evita di doverli raddoppiare. Verificato: cercare `%` restituisce 0 righe |
| Preset temporali calcolati su `Europe/Rome` con `Intl` | Corretti anche al cambio dell'ora legale, senza librerie di date. Il file è uno solo: `lib/events/timezone.ts` |
| Il **centro** del raggio va nell'URL (`lat` / `lng`) | Un raggio ha senso solo con un centro: così il filtro resta condivisibile. La posizione precisa resta dietro "Vicino a me" |
| Suggerimenti **dal nostro database**, non da un geocoder esterno | Coerenza con "niente dati finti", nessun costo, nessun servizio da configurare |
| `scroll: false` su ogni navigazione di filtro | Chi sta filtrando guarda i filtri: la pagina non deve tornare in cima |

# Deploy e rilascio

1. Il codice vive su GitHub (`openstagemusic/openstage`, ramo `main`).
2. Ogni push su `main` fa partire automaticamente un **deploy di produzione** su Vercel.
3. Progetto Vercel: `openstage` (team `ceskobeatzmusic-2476`), piano Hobby.
4. **Il preset del framework deve restare "Next.js"**: una volta è stato impostato su "Other" e i deploy si rompevano. Non va mai reintrodotto quell'override.
5. La protezione "Vercel Authentication" (Deployment Protection) è **disattivata**, altrimenti il sito pubblico chiederebbe un login Vercel.

**URL di produzione:** `https://openstage-alpha.vercel.app`

## Checklist di fine fase

1. `npm run build` senza errori.
2. Verifica delle regole nel database (SQL di collaudo qui sopra).
3. Verifica delle pagine in produzione, anche da utente non loggato (i percorsi protetti devono rimandare al login).
4. Commit su Git chiamato **"Fase N"** e push.
5. `docs/review-fase-N.md` aggiornato (cosa funziona, suggerimenti, debito tecnico, cose da riprendere).
6. Se la fase ha cambiato qualcosa di stabile (architettura, variabili, regole, schermate): `npm run manuale` per rigenerare questo PDF, con versione e data nuove in `meta.json`.

# Errori già incontrati e come si risolvono

| Sintomo | Causa | Rimedio |
|---|---|---|
| Il deploy su Vercel si rompe senza un errore di codice | Preset del framework impostato su "Other" | Riportare il preset a **Next.js** e ricreare il deploy di produzione |
| In locale la build va, su Vercel no (o viceversa) | Variabile d'ambiente mancante in uno dei tre ambienti | Impostarla su Production, Preview e Development |
| Login/registrazione dall'API falliscono con errore captcha | Turnstile è attivo su Supabase: serve un token vero | Provare da browser, non con chiamate dirette. Il server deve chiamare `siteverify`; Supabase lo fa se CAPTCHA è attivo |
| `supabase migration list` non mostra le migrazioni | Manca la riga in `supabase_migrations.schema_migrations` | Inserire la voce `(version, name)` corrispondente al file |
| Un test RLS "mente" (una modifica bloccata sembra passata) | I claim JWT erano rimasti attivi da un test precedente | `set local request.jwt.claims = ''` prima del test |
| L'URL di autorizzazione OAuth arriva rotto (`client_id: invalid`) | I parametri `&` vengono corrotti aprendolo da terminale/chat | Incollare l'URL a mano nella barra degli indirizzi |
| Il selettore di lingua non cambia nulla | La scelta manuale è nel cookie `NEXT_LOCALE` | Svuotare il cookie o usare lo switcher (non l'URL) |
| L'email di conferma o di reset non arriva | SMTP o dominio Resend non configurati, oppure email in spam | Controllare Resend → Domains (verified), Supabase → Auth → SMTP, e la cartella spam |
| Il link di reset porta al login o dice "scaduto" | Il link usa ancora il flusso PKCE, che dipende dal browser della richiesta, oppure l'URL non è tra i Redirect URLs | Usare i template Supabase con `token_hash` verso `/auth/confirm`; in Supabase → Auth → URL Configuration aggiungere `https://getopenstage.it/**`, `https://getopenstage.it/reset-password`, `https://getopenstage.it/auth/callback`, `http://localhost:3000/**` |
| Google torna alla Home da scollegato | I cookie di sessione non vengono applicati alla redirect del callback | Il callback deve impostare i cookie sulla `NextResponse.redirect` (non solo su `cookies()`); verificare anche i Redirect URLs Supabase |
| La lista degli eventi in Home è **sempre vuota** | Una costante importata da un modulo `"use client"` dentro un Server Component arriva `undefined` | Tenere le costanti condivise in un file neutro: `lib/pagination.ts`. **Regola**: mai importare valori da un modulo `"use client"` in un componente server |

| Sintomo | Causa | Rimedio |
|---|---|---|
| `?page=2` mostra una pagina vuota | Il numero di pagina richiesto supera l'ultima disponibile | Riportare la pagina all'ultimo valore valido invece di mostrare il vuoto |
| Il pop-up di un locale con più serate esce dalla mappa e si taglia | La scheda cresce con il numero di date, e il pop-up desktop sta **sopra** il pin | Una serata alla volta con le frecce (altezza costante) + margine della mappa dal lato giusto |
| La pagina torna in cima usando ricerca e filtri | Navigazione senza `scroll: false`; i menu a tendina nativi su mobile possono saltare in cima comunque | `scroll: false` su ogni navigazione + `useScrollAnchor`, che ripristina la posizione solo se il browser ha davvero saltato |
| Un blocco SQL con `union all` si interrompe | I valori di tipo enum vanno convertiti esplicitamente nelle unioni | Scrivere `'published'::event_status` e simili |
| La build fallisce con `useSearchParams` | In Next 15 un componente client che legge i parametri va avvolto in `<Suspense>` | Avvolgere il componente (o il suo gruppo) in un confine `<Suspense>` |
| La mappa resta grigia / il worker non si carica | Il worker di MapLibre non è raggiungibile dal percorso previsto | Servirlo dal nostro dominio: `scripts/copy-maplibre-worker.mjs` lo copia in `public/` durante `prebuild` |
| Il titolo-poster appare tagliato in basso | Con `line-height` molto sotto 1 il glifo esce dalla line box e viene ritagliato | Usare `leading-none` (1) e recuperare la compattezza con un margine negativo, non con un `line-height` più piccolo |
| Un link condiviso su `/eventi/<uuid>` non apre la scheda | Dalla Fase 3 l'URL canonico è `/eventi/[città]/[slug]` | Il vecchio indirizzo fa un redirect permanente verso quello nuovo: usare sempre il link canonico |

# Come si verifica una pagina (browser headless)

Le fasi si collaudano in un **browser vero**, non solo con la build: si usa il Chrome installato sul computer, guidato da `puppeteer-core` (nessun browser da scaricare). Lo script temporaneo si crea, si usa e **si cancella**: non resta nel repository.

La ricetta messa a punto nella Fase 2:

1. `puppeteer.launch({ executablePath: "...chrome.exe", headless: true })`.
2. `page.setViewport()` con **almeno due misure**: desktop 1280×900 e mobile 390×844 (`isMobile`, `hasTouch`), più 320 px per scoprire l'overflow più stretto.
3. `page.goto(..., { waitUntil: "networkidle2" })` e una pausa: la mappa ha bisogno di qualche secondo prima di essere affidabile.
4. Per cliccare **senza falsare il test**: leggere `getBoundingClientRect()` dell'elemento e usare `page.mouse.click(x, y)` / `page.touchscreen.tap(x, y)`. `elementHandle.click()` fa scorrere l'elemento in vista e nasconde proprio i difetti di scroll che si vogliono trovare.
5. Misurare i **numeri**, non le impressioni: `window.scrollY` prima e dopo, il rettangolo del pop-up confrontato con quello della mappa, il conteggio dei risultati.
6. Ascoltare `console` (errori) e `pageerror`: una fase non è chiusa se la console non è pulita.
7. Provare **in locale e in produzione**, e in **entrambe le lingue**. In locale mancano gli header geografici di Vercel: la lingua parte in inglese e il centro mappa è Roma. È utile per accorgersi di differenze tra i due ambienti.

> I pin della mappa sono disegnati su WebGL e **non esistono come elementi HTML**: per cliccarli si recupera l'istanza di MapLibre dai dati interni di React e si usa `map.project([lng, lat])` per sapere dove si trova sullo schermo.

# Il manuale PDF

Questo documento **si genera**, non si scrive a mano:

- i contenuti sono file Markdown in `docs/manuale/` (uno per capitolo/macro-area);
- `docs/manuale/meta.json` contiene titolo, versione, data e quali fasi copre;
- `npm run manuale` costruisce il PDF con il Chrome installato sul computer (nessun servizio esterno, nessun browser da scaricare) e lo salva come `docs/manuale/OpenStage-Manuale.pdf`;
- l'indice si costruisce **da solo** dai titoli dei file `.md`.

Per aggiornarlo: si modificano i `.md`, si aggiornano versione e data in `meta.json`, si rilancia `npm run manuale` e si committa il PDF. Il generatore è `scripts/manuale.mjs` e **non** entra né nella build di Next.js né nel deploy: serve solo in locale.

# Roadmap delle fasi

| Fase | Contenuto previsto |
|---|---|
| 1 · 1.2 · 1.3 | ✅ Fondamenta, indurimento del database, ruoli definitivi |
| 2 | ✅ Scoperta: mappa, pagine evento, ricerca, filtri, salvataggio, vetrina |
| 3 | ✅ Directory artisti/locali, reclamo, dashboard, editor eventi, condivisione, preferiti |
| 4 | ✅ Qualità dei dati, serate collegate, segnalazioni e moderazione, serate passate |
| 5.4 | ✅ Remake grafico della Home: muro di locandine, lista a righe, mappa a tutta pagina, guscio |
| 5.5A · 5.5B | ✅ Restyling login/registrazione/account/onboarding e scheda evento + popup/bottom sheet mappa |
| 6 | ✅ Pagine pubbliche ed editoriali, `/citta` e schede città, modulo contatti reale e coda messaggi admin; sezione Città in Home e Dashboard evidenziata |
| 7A | ✅ Email transazionali (Resend), conferma email, reset password, Turnstile lato Supabase. Resta `B48` (risposta ai contatti) |
| 7B | Area fan, notifiche in-app, coda email, webhook, digest. **Prossima dopo la 5.5** |
| 8 | Pannello admin completo, 2FA e claim a invito |
| 9 | Community: "segui" e inviti virali |
| 10 | Spotify e area artista |
| 11 | Monetizzazione opzionale (PayPal): richiede Vercel Pro e partita fiscale |
| 12 | Serie, locali chiusi, bacheca ingaggi |
| 13 | SEO, legal, audit, polish finale e remake parte 2 |

> L'ordine dalla 6 in poi è indicativo e viene dettagliato una fase per volta, con un piano scritto prima di scrivere codice. La roadmap approvata è in `docs/roadmap.md`; se questo manuale dice altro, vale la roadmap.

# Diario delle fasi

Questa è la memoria del progetto: per ogni fase, cosa è stato costruito, quali decisioni sono state prese e perché, cosa è stato rimandato. Serve per non dover ricostruire a posteriori il ragionamento dietro una scelta.

## Fase 1 — Fondamenta

**Obiettivo:** mettere in piedi la piattaforma "vera" fin dal primo giorno: database reale, accesso reale, niente dati finti o schermate di esempio.

**Cosa è stato costruito**

- Progetto Next.js 15 (App Router, TypeScript) con Tailwind v4, tutte le pagine pubbliche renderizzate lato server.
- **Design system** dark cyber-neon: sfondo `#07070D`, accenti viola/ciano/rosa, font Space Grotesk + Inter, navbar e footer in vetro, card con gradiente.
- **Doppia lingua IT/EN** con dizionari modulari, scelta automatica in base al paese e selettore manuale ricordato nel cookie `NEXT_LOCALE`.
- **Accesso reale** con Supabase: registrazione email + password, accesso con Google, captcha Cloudflare Turnstile, gestione della sessione con cookie (`@supabase/ssr`) rinnovata dal middleware.
- **Ruoli** (fan / artista / locale / admin) scelti in registrazione e memorizzati nel profilo.
- Pagine: Home, Artisti, Locali, Per gli Artisti, Per le Venue, Chi siamo, Contatti, Privacy, Termini, Cookie, più l'elenco di artisti e locali **letto dal database** con paginazione.
- **Player Spotify** predisposto in fondo a ogni pagina: il componente è montato nel layout, ma resta **spento** finché non viene configurata una playlist pubblica (`NEXT_PUBLIC_SPOTIFY_PLAYLIST_ID`), perché non deve mostrare player vuoti o contenuti finti. Navigazione mobile a tendina dalla barra in alto (**non** esiste una barra fissa in basso).
- Prima migrazione SQL (`0001_init.sql`) con tabelle, RLS e trigger.
- Repository GitHub + deploy continuo su Vercel.

**Decisioni e perché**

- **Supabase scelto** rispetto a Neon+Clerk, Nhost, Appwrite, Firebase e Turso: è l'unico che unisce Postgres puro e Row Level Security legata all'utente loggato.
- **La scelta del ruolo sta dopo l'accesso con Google**, non nell'URL di ritorno: passare `?role=` avrebbe significato fidarsi di un parametro che il browser può cambiare.
- **Conferma email disattivata** in questa fase, per poter provare subito i flussi senza un servizio di invio email già configurato.
- Elenchi con **paginazione a 12 elementi** e query fatte sul database (nessun dato di esempio).

**Verifiche superate:** build di produzione senza errori; pagine pubbliche 200 in produzione; i percorsi protetti rimandano al login; ruoli e profili verificati via SQL.

**Rimandato:** mappa, eventi reali, schede artista/locale complete, ricerca, email.

## Fase 1.2 — Indurimento del database

**Obiettivo:** sistemare il database *prima* che contenga dati veri, quando correggere costa poco.

**Cosa è stato costruito** (`0003_hardening.sql`)

- `search_path` fissato sulla funzione che aggiorna `updated_at` (evita di eseguire funzioni sbagliate per errore).
- Revoca dell'esecuzione diretta delle funzioni interne di registrazione e di controllo ruolo.
- **3 indici sulle chiavi esterne** di `events` (le ricerche future ne hanno bisogno).

- **12 policy riscritte** con `(select auth.uid())`, che Postgres valuta una volta sola per query invece che riga per riga: è il modo corretto e più veloce.
- **Policy duplicate unite** su `profiles` (da 18 a 16 policy totali).
- **Storico migrazioni regolarizzato**: le voci `0001` … `0003` allineate ai nomi dei file, così gli strumenti Supabase e il repository raccontano la stessa storia.

**Verifiche superate:** il trigger di registrazione crea ancora il profilo; l'utente può cambiare il proprio ruolo ma non promuoversi ad admin; un utente normale vede una sola riga di `profiles`; i visitatori anonimi non vedono profili né impostazioni; gli avvisi di performance (`auth_rls_initplan`, indici mancanti, policy multiple) sono spariti.

**Rimandato:** attivazione della protezione "password compromesse" (richiede un intervento in dashboard) e la revoca di alcuni token di servizio usati durante la configurazione.

## Fase 1.3 — Ruoli definitivi

**Obiettivo:** una volta scelto, il ruolo non si cambia più; solo un fan può diventare artista o locale, una volta sola.

**Cosa è stato costruito** (`0004_role_lock.sql` + interfaccia)

- **Nuova regola nel database**: l'unica transizione consentita è `fan → artista | locale`. Da artista o locale non si torna indietro, e non ci si promuove ad admin.
- **La RLS chiede il ruolo coerente**: per creare una riga `artists` bisogna essere artista, per `venues` bisogna essere locale, per un evento bisogna essere artista o locale. Un fan che prova a farlo via API viene **respinto dal database**, non solo nascosto dall'interfaccia.
- **Correzione di un difetto ereditato dalla Fase 1**: il trigger annullava anche l'assegnazione del ruolo `admin` al profilo dello staff (durante la registrazione non esistono i dati di sessione). Ora funziona e l'account dello staff è allineato.
- **Interfaccia**: badge colorato del ruolo nella barra in alto, link contestuale ("Diventa artista o locale" per un fan, "Crea evento" per artista/locale), pagina di scelta del ruolo riservata ai fan (chi è già artista o locale vede "Il tuo ruolo è già attivo"), pagine protette per ruolo.
- **Pulsante di scelta più onesto**: avvisa che la scelta di Artista/Locale è definitiva e, dopo il salvataggio, **rilegge dal database** il ruolo effettivo: se il database non l'ha accettato, l'utente vede un errore invece di una finta riuscita.
- **Testo corretto**: il sottotitolo della scelta del ruolo diceva "potrai cambiarlo quando vuoi", che con le nuove regole era falso.
- Helper riutilizzabili: `lib/auth/roles.ts` (permessi) e `lib/supabase/session.ts` (utente + ruolo in una sola lettura per richiesta).

**Verifiche superate:** artista → locale/fan/admin annullati; locale → fan annullato; fan → artista e fan → locale riusciti e poi bloccati; fan respinto (errore 42501) quando prova a creare righe artista o eventi; artista autorizzato; correzione da parte dell'admin possibile; build e produzione verdi; nessun peggioramento negli avvisi di sicurezza.

**Rimandato:** nulla di bloccante. Le regole verranno estese alle future tabelle (per esempio ai preferiti) man mano che nascono.

## Fase 2 — Scoperta degli eventi

**Obiettivo:** rendere la piattaforma utile per ciò che serve davvero — trovare il concerto di stasera — con dati reali, nessuna schermata di esempio.

**Cosa è stato costruito** (tre blocchi: 2.1, 2.2, 2.3)

*2.1 — Mappa ed eventi*

- **Mappa interattiva** con MapLibre GL JS e **OpenFreeMap**: nessuna chiave API, nessun account, uso commerciale consentito, attribuzione obbligatoria. L'indirizzo dello stile è isolato in `lib/map/config.ts`.
- **Globo 3D** con la proiezione nativa di MapLibre e una rotazione introduttiva che parte solo su desktop e solo se il sistema non chiede "meno animazioni".
- **Pin degli eventi raggruppati per locale**: un locale con più serate ha un pin solo con il numero delle date, altrimenti i pin si sovrappongono e solo l'ultimo è cliccabile.
- **Scheda del pin** con data e ora nel fuso dell'evento, titolo, artista, locale, genere, tipo, ingresso gratuito e i pulsanti *Pagina evento*, *Biglietti*, *Apri in Google Maps*, *Salva*, *Condividi*.
- **Posizione approssimata**: un evento può dichiarare un'area invece di un indirizzo esatto, e la pagina evento lo spiega.
- **Pagina evento** `/eventi/[id]` con rendering lato server, dati strutturati e anteprima social; il titolo della serata è quello ufficiale della fonte, e la riga dell'artista sparisce quando coincide con il titolo.
- **Modalità a tutto schermo** `/mappa` con gli stessi filtri della Home.
- **Gesture mobile native**: un dito scorre la pagina, la mappa si muove solo a tutto schermo o con due dita.
- **Salvataggio reale** degli eventi (`saved_events` + RLS + server action).
- **Artisti senza account**: `artists.profile_id` diventa nullable, simmetrico ai locali. Serve per gli eventi reali inseriti dalla redazione.

*2.2 — Ricerca e filtri*

- **Barra di ricerca** con suggerimenti raggruppati (città, artisti, locali, eventi, generi) presi dal **nostro** database: nessun geocoder esterno.
- **Filtri rapidi**: stasera, questo weekend, questa settimana, prossimi 30 giorni, raggio in km, genere, tipo di evento, ingresso gratuito.
- **Tutto lo stato vive nell'URL**: un link filtrato è condivisibile, la pagina si compone lato server, e lista e mappa leggono gli stessi parametri.
- **Ricerca e filtri in una funzione SQL** (`search_events`), non con filtri lato applicazione: ricerca testuale su più tabelle e paginazione corretta *con la distanza*.
- **Preset temporali su** `Europe/Rome` calcolati con `Intl`: corretti anche al cambio dell'ora legale, senza librerie di date.
- Il **centro del raggio** va nell'URL: un raggio ha senso solo con un centro, e così resta condivisibile. La posizione precisa resta dietro "Vicino a me".

*2.3 — Vetrina*

- **Schede / Tabella**: la stessa lista in due modi, con paginazione a 12 e i filtri mantenuti cambiando pagina.
- **"In evidenza"**: fino a tre serate scelte a mano dallo staff, **mai pagate**, con una scadenza reale (`featured_until`). La sezione resta nascosta finché non c'è almeno una serata scelta, e sparisce mentre si filtra.
- **Bento grid con contatori reali** (eventi, artisti, locali, città): i numeri sono scritti nel HTML dal server, quindi veri anche senza JavaScript; l'animazione è solo un di più.
- **Stato vuoto** che invita a segnalare locali e artisti quando il database è ancora piccolo.

**Decisioni e perché**

- **OpenFreeMap invece di MapTiler**: MapTiler richiede account e chiave API; OpenFreeMap no, consente l'uso commerciale e si "paga" con l'attribuzione. Rischio accettato: è un'istanza finanziata da donazioni, senza garanzie di servizio. Mitigazione: l'URL dello stile è in un punto solo.
- **Niente Globe.gl**: la proiezione globo è nativa in MapLibre, quindi zero dipendenze in più.
- **Un pin per locale**: tre concerti nella stessa sala erano tre pin sovrapposti.
- **I suggerimenti vengono dal nostro database**: coerenza con "niente dati finti", nessun costo.
- **Filtri nel database, non nell'app**: con i filtri lato client la paginazione con la distanza sarebbe sbagliata.
- **"In evidenza" non è pubblicità**: le serate le sceglie lo staff, non si può pagare per comparire.

- **Eventi reali inseriti uno alla volta**, mai inventati. Quando li inserisce la redazione, l'evento è marcato `is_unclaimed` finché il locale o l'artista non reclamano la scheda.

**Verifiche superate:** build verde; verifica in browser reale (Chrome headless) **in locale e in produzione**, in italiano e in inglese, desktop 1440 px e mobile 390 px; ricerca, filtri e raggio provati con i dati reali; RLS provata impersonando gli utenti; scroll della pagina conservato quando si filtrano i risultati; zero errori in console.

**Correzioni dopo la chiusura della fase** (segnalate provando la produzione)

- Il **pop-up di un locale con più serate usciva dalla mappa** e si tagliava: ora le serate si scorrono **una alla volta** con le frecce numerate `‹ 1 / 3 ›` e la scheda ha altezza costante; il margine della mappa va dal lato giusto (in alto per il pop-up desktop, in basso per il pannello mobile, che ha anche un tetto di `70vh`).
- **La pagina tornava in cima** usando ricerca e filtri: la causa principale era una versione precedente senza `scroll: false`. Restavano due casi reali, sistemati: il pulsante "Azzera i filtri" non aveva `scroll: false`, e i menu a tendina nativi su mobile possono saltare in cima comunque. Ora c'è `useScrollAnchor`, che ripristina la posizione **solo** se il browser ha davvero saltato.

**Rimandato:** pagina dei preferiti (B19), fuso del visitatore nei preset temporali (B20), indici trigram nella ricerca (B21), test automatici end-to-end (B05), editor degli eventi (Fase 4).

## Fase 3 — Directory, reclami, editor e condivisione

**Obiettivo:** trasformare le schede e l'archivio in uno strumento usabile da artisti e locali veri, non solo dalla redazione.

**Cosa è stato costruito** (blocchi 3.0 → 3.4)

- **Lingua negli URL**: `/it/...` e `/en/...`, `canonical`, `hreflang`, `sitemap.xml` e `robots.txt`. Il cookie resta solo come preferenza per il primo redirect. Senza questo passaggio la versione italiana non sarebbe indicizzabile.
- **Directory reali** di artisti e locali: ricerca, filtri, paginazione, schede con slug, concerti futuri, JSON-LD, link crawlable.
- **Reclamo della scheda**: l'artista dimostra la proprietà con un link pubblico e viene collegato subito; il locale invia una richiesta che l'admin approva o rifiuta. Una scheda per profilo, una richiesta in attesa alla volta, possibilità di ritirarla.
- **Verifica dello staff**: il badge "Verificato" non è decorativo, è il permesso di pubblicare. Il primo evento di un account nuovo può passare dalla revisione.
- **Dashboard artisti/locali** con panoramica, "I miei eventi", creazione, preferiti, profilo e account.
- **Editor eventi**: bozza/pubblicazione/annullamento, locandina caricata o generata, posizione esatta o approssimata, pin sulla mappa, prezzo libero, link biglietti, collegamento all'artista, avviso doppioni e tetto anti-spam di 10 serate nelle 24 ore.
- **Pagina evento completa**: locandina intera, countdown, "Aggiungi al calendario" (Google e `.ics`), condivisione con Share Card 1080×1920 e 1080×1080 e QR, anteprima Open Graph disegnata dal server. Una serata annullata lo dice e il suo `.ics` diventa `CANCELLED`.
- **Pagina dei preferiti** in dashboard.
- **Artisti senza account e locali senza account** possono esistere; la nota "inserito da OpenStage" si calcola dall'assenza di reclamo, non è più una colonna salvata.

**Decisioni e perché**

- La lingua **deve** stare nell'URL: con un cookie Googlebot riceverebbe sempre l'inglese e la directory italiana non esisterebbe per la ricerca.
- **Photon** per l'autocomplete degli indirizzi: nessuna chiave, nessun account; MapTiler avrebbe richiesto un piano commerciale.
- **"Verificato", non "Pro"**: "Pro" collide con il principio "gratis per sempre" e non esiste ancora un piano a pagamento.

- **Unione dei doppioni solo dall'admin**: due set nello stesso locale la stessa sera sono legittimi, quindi nessun blocco automatico; l'admin unisce e può disfare.
- **QR e Share Card disegnati in locale** con `qrcode` e canvas: nessun servizio esterno, nessun costo per immagine.

**Verifiche superate:** build e produzione verdi; prove in browser con sessioni reali; reclamo artista e locale, coda admin, creazione e modifica eventi, `.ics`, Share Card, QR confrontato modulo per modulo, pagina evento in italiano e inglese, mobile e desktop.

**Rimandato:** email e notifiche (Fase 7), password dimenticata (B36), conferma email (B35), cancellazione account (B07), rifinitura grafica delle pagine pubbliche (Fase 6/13).

## Fase 4 — Qualità dei dati e moderazione

**Obiettivo:** evitare che una serata vera venga persa o che una falsa resti online per sempre, senza punire un intero catalogo.

**Cosa è stato costruito**

- **Città normalizzata** (`city_key` generata dal database): "Milano", "milano" e "Milano (MI)" sono la stessa città nei filtri e nei contatori.
- **Raggio onesto**: per una serata approssimata la distanza si misura dal **bordo** della zona, non solo dal centro.
- **Controllo del pin**: geocodifica inversa tri-stato (`found` / `empty` / `unavailable`); pin sul mare rifiutato, pin lontano dalla città dichiarata avvisato o bloccato, Photon irraggiungibile non blocca nessuno.
- **"Come arrivare"** per cortili, centri sociali e luoghi senza indirizzo perfetto.
- **Serate collegate**: il locale può collegare una scheda artista esistente; un artista non può attribuirsi una serata che non è sua. Avviso doppioni con "Crea comunque".
- **Unione dei doppioni** dall'admin, reversibile, con i Preferiti spostati sulla scheda buona e redirect permanente dalla scheda unita.
- **Segnalazioni**: "Segnala questa serata" per chi ha un accesso, una volta per persona, motivi chiusi e nota. Alla terza segnalazione da persone diverse e verificate la serata va in pausa automaticamente.
- **Pannello** `/admin/eventi`: serate online, in pausa e segnalate; annulla, rimetti online, torna in bozza, sospendi o riattiva l'autore, unisci un doppione.
- **Serate passate**: sezione dedicata per artista, locale e dashboard; eliminazione definitiva per il proprietario e per l'admin, con conferma.

**Decisioni e perché**

- **La pausa è per serata, non per autore**: punire tutto il catalogo per una serata contestata sarebbe arbitrario. L'account si sospende solo a mano, davanti a un comportamento sistematico.
- **Nessuna recensione pubblica**: la segnalazione è uno strumento privato di moderazione, non un voto. Fedele al principio "niente metriche vanitose".
- **Eliminazione definitiva** solo per proprietario e admin, dopo conferma: una serata sbagliata non deve restare per sempre, ma non deve nemmeno sparire per un click distratto.
- **Le regole nel database**: il tetto anti-spam, il blocco delle transizioni di ruolo e la pausa automatica non dipendono dal form.

**Verifiche superate:** build e produzione verdi; prove con utenti Auth reali; città deduplicate; raggio con eventi gemelli; pin sul mare e a Berlino; serata collegata e doppione; unione e separazione; segnalazioni, pausa, rimessa online e annullamento; pulizia delle fixture dopo i test.

**Rimandato:** notifiche email delle decisioni (Fase 7), pulsante "Unisci" provato via RPC e non ancora cliccato in browser, sospensione autore resa visibile ma non cliccata.

## Fase 5.4 — Home poster wall

**Obiettivo:** dare alla Home un'identità editoriale e underground, senza trasformare OpenStage in una vetrina pubblicitaria.

**Cosa è stato costruito**

- **Muro di locandine**: fino a quattro copertine reali, con ingresso sfalsato; nessuna immagine di repertorio, nessuna card.
- **Titolo-poster** "Stasera si suona." con Anton e IBM Plex Mono per etichette e metadati.
- **Ticker** con i valori del progetto: entra una volta e si ferma, non è un marquee.
- **Lista a righe**: data grande, titolo, artista/locale, genere, orario e ingresso.
- **Ricerca e filtri editoriali** senza pillole né glass, solo in Home; `/mappa` mantiene la variante precedente.
- **Mappa a tutta pagina**: fascia con intestazione, pin e link alla modalità full screen.
- **Cos'è OpenStage**: statement, tre punti e il claim "Gratis. Per sempre. Per chi suona.".
- **Come funziona** senza bento grid: tre passi, contatori reali e le due porte artista/locale.
- **Guscio coerente**: navbar, footer, language switcher e paginazione con bordi netti, etichette mono e CTA rettangolari.
- **Movimento sobrio**: entrata del titolo, stagger delle locandine, reveal con `IntersectionObserver`, hover delle righe e puntino live. Tutto CSS, tutto disattivato con `prefers-reduced-motion`; nessuna dipendenza nuova.

**Decisioni e perché**

- **Concept 01 "Muro di locandine"**: scelto dal proprietario; i concept alternativi sono superati.
- **Niente Framer Motion**: per un fade aggiungeva ~45 KB gzip; resta in Fase 13 per le interazioni complesse.
- **Niente card e niente glass in Home**: la Home deve sembrare una locandina appesa, non una dashboard.
- **La Home non ha più l'interruttore schede/tabella**: la lista è a righe; la mappa ha la sua pagina.
- **Nessun nuovo servizio, variabile o migration**: il lavoro è tutto di interfaccia e tipografia.
- **Il ticker non loopa**: un'animazione infinita sarebbe rumore, non identità.

**Verifiche superate:** build verde; nessun overflow a 320, 390 e 1280 px; italiano e inglese; menu mobile; ricerca, filtri, mappa e paginazione; `prefers-reduced-motion` verificato con zero animazioni; H1 unico e leggibile; produzione Vercel aggiornata con commit `Fase 5.4`.

**Rimandato:** il resto delle pagine pubbliche (Artisti/Locali usano ancora lo stile precedente) passa alla Fase 6; la seconda parte delle micro-interazioni alla Fase 13.

## Fase 6 — Fiducia pubblica e densità locale

**Obiettivo:** sostituire i segnaposto con pagine pubbliche vere e rendere OpenStage trovabile città per città, senza aggiungere servizi esterni e senza toccare il modello di business.

**Cosa è stato costruito**

- **6.1 — Pagine pubbliche:** `/about`, `/per-gli-artisti`, `/per-le-venue`, `/contatti` e `/faq` con contenuti reali in italiano e inglese, metadata, canonical, JSON-LD e footer con link coerenti. Social e playlist restano nascosti finché non ci sono URL veri.
- **Modulo contatti reale:** scrive in `contact_messages` con validazione, honeypot e limiti di lunghezza; nessun servizio email in questa fase. La coda è in `/admin/messaggi`, con stati `new`, `read`, `closed` e risposta via `mailto:`.
- **6.2 — Città:** `/citta` e `/citta/[city]` generate dagli eventi reali, con prossime serate, artisti, locali, link alla mappa filtrata, canonical e JSON-LD. Soglia editoriale di tre serate future: sotto soglia la pagina resta raggiungibile ma è `noindex` e fuori dalla sitemap.

- **6.3 — Moderazione minima:** migration `0028_contact_messages.sql` con tabella, indici e RLS (inserimento pubblico, lettura e gestione solo admin); route admin senza sessione protetta.
- **Rifinitura post-6.3:** nuova sezione **Città** in Home (massimo sei città, ordine alfabetico, conteggio serate e prossima data, link a `/citta`) e pulsante **Dashboard** evidenziato nella navbar desktop.

**Decisioni e perché**

- **Nessun servizio nuovo:** la fase usa solo Supabase già configurato; Resend arriva con la Fase 7.
- **Nessun dato finto:** le città e i messaggi vengono dal database; il footer non mostra social o playlist senza URL reali.
- **Soglia a tre serate:** evita pagine città sottili e duplicate, senza nascondere quelle esistenti.
- **Home corta:** la sezione Città non deve diventare un elenco infinito; l'indice completo vive su `/citta`.
- **Risposta ai messaggi con `mailto:`:** sufficiente finché non esiste la casella ufficiale; la risposta integrata è rimandata alla Fase 7.

**Verifiche superate:** build verde; pagine pubbliche 200 in produzione; form contatti con invio valido, honeypot ed errore; messaggio visibile all'admin e route protetta senza sessione (404); città con zero, una e più serate; `noindex` sotto soglia; italiano e inglese; controlli Lighthouse su Home, città e contatti.

**Rimandato:** email transazionali, conferma email e reset password (Fase 7); risposta integrata ai messaggi (`B48`); alcune voci di audit (`B08`, `B26`, `B28`, `B43`).

## Fase 5.5A - Coerenza visiva: accesso e account

**Obiettivo:** riportare login, registrazione, account e onboarding al linguaggio della Home 5.4 senza cambiare la logica.

**Cosa è stato costruito**

- Nuovo guscio editoriale condiviso `AuthShell`: titolo-poster a sinistra, pannello sobrio a destra, etichette mono e valori reali del prodotto.
- Stili condivisi per input, label, pulsanti ed errori (`components/auth/formStyles.ts`).
- Restyling di `/login`, `/register`, `/account` e delle pagine di onboarding artista/locale.
- Ruolo rimosso dal form di registrazione: si sceglie **dopo** la conferma email, così il profilo non nasce con un ruolo scelto per errore.

**Verifiche superate:** build verde; layout a 320, 390 e 1280 px; italiano e inglese; form reali; Turnstile presente; screenshot desktop/mobile controllati.

## Fase 7A - Email reale, conferma e recupero password

**Obiettivo:** chiudere l'accesso affidabile con dominio, Resend, SMTP Supabase e Turnstile verificato lato server.

**Cosa è stato costruito**

- Dominio `getopenstage.it` su Aruba per la ricezione; Resend `eu-west-1` per l'invio.
- Caselle `noreply@getopenstage.it` e `ciao@getopenstage.it`; SMTP Supabase configurato.
- Conferma email obbligatoria; dopo la conferma si sceglie il ruolo.
- Link **Password dimenticata?**, pagina `/forgot-password`, pagina `/reset-password` e aggiornamento password con Supabase.
- CAPTCHA Supabase attivo: le richieste senza token vengono rifiutate e il server chiama `siteverify`. Il widget ha action `signup`, `login`, `reset_password` e si resetta dopo un errore.
- Callback OAuth corretto: i cookie di sessione vengono applicati alla redirect response.
- Conferma e reset cross-browser: template Supabase con `token_hash` verso `/auth/confirm`, così il link funziona anche se Gmail lo apre in un browser diverso.

- Dopo il login, chi non ha ancora scelto il ruolo viene mandato a `/onboarding/role`.

**Decisioni e perché**

- **Aruba riceve, Resend invia:** si tengono separati ricezione e invio transazionale.
- **Ruolo dopo la conferma:** un'email sbagliata non deve creare un profilo artista/locale.
- **Nessuna email per Google:** Google ha già verificato l'indirizzo; resta solo la scelta del ruolo.
- **Turnstile lato Supabase:** evita una seconda verifica e sfrutta il backend di auth già esistente.

**Verifiche superate:** email di conferma ricevuta; reset password funzionante; Google con ruolo salvato; signup senza token rifiutato con `captcha_failed`; build e deploy verdi.

**Rimandato:** risposta integrata ai messaggi (`B48`) e cambio email dal profilo.

## Fase 5.5B - Scheda evento e popup mappa

**Obiettivo:** portare scheda evento e popup/bottom sheet al linguaggio editoriale, senza nuove funzioni.

**Cosa è stato costruito**

- Scheda evento copertina-first, titolo-poster, badge mono, metadati in righe separate e azioni in fondo (larghezza piena su mobile).
- Popup desktop compatto con bordo `line-strong`, fondo scuro e gerarchia chiara; margine superiore aumentato per non finire sotto ricerca e filtri.
- Bottom sheet mobile con copertina più grande, chiusura esplicita e target ≥ 44 px.
- Gruppo locale con una serata alla volta, frecce e contatore.
- Pulsanti calendario, condivisione, salvataggio e segnalazione allineati al nuovo stile.

**Verifiche superate:** build verde; screenshot desktop/mobile della scheda evento; popup desktop aperto su un pin reale; italiano e inglese; nessuna modifica a database o RLS.

**Rimandato:** directory artisti/locali (5.5C), guscio dashboard (5.5D) ed editor eventi (5.5E).

## Fase 5.5C - Directory artisti e locali

**Obiettivo:** riportare `/artisti`, `/locali` e le schede al linguaggio editoriale della Home 5.4, senza toccare filtri, query o dati.

**Cosa è stato costruito**

- Testata editoriale con etichetta mono, titolo-poster e introduzione.
- Filtri squadrati e visibili: ricerca, città, genere per gli artisti; contatore risultati in mono.
- Card con monogramma tipografico grande, nome leggibile e città/genere o indirizzo in etichetta mono; badge verificato/non reclamato discreto.
- Schede artista/locale con monogramma, nome-poster, metadati in righe `border-line`, claim panel sobrio ed eventi in arrivo/passati con card editoriale.
- `EventTeaser` allineato al nuovo linguaggio, riusato anche nei Preferiti.

**Verifiche superate:** build verde; screenshot desktop di `/artisti`, `/locali`, `/artisti/[slug]` e `/locali/[slug]`; layout mobile delle directory; filtri e paginazione invariati; italiano e inglese; nessuna modifica a database o RLS.

**Rimandato:** guscio dashboard (5.5D) ed editor eventi (5.5E).

## Come si aggiorna questo diario

A ogni fine fase si aggiunge una sezione con lo stesso schema: obiettivo, cosa è stato costruito, decisioni e perché, verifiche superate, cosa è rimandato. La review di fase (`docs/review-fase-N.md`) resta il posto dove finiscono giudizi, suggerimenti e debito tecnico da riprendere: qui va solo ciò che è stato deciso e fatto.

# Changelog del manuale

| Versione | Data | Fasi coperte | Novità |
|---|---|---|---|
| 1.0 | 16 ottobre 2026 | 1 · 1.2 · 1.3 | Prima edizione: guida utente, manuale tecnico (stack, database, sicurezza, deploy, errori noti), diario delle fasi |
| 1.1 | 16 ottobre 2026 | 1 · 1.2 · 1.3 · 2 | Fase 2: mappa con globo e pin, pagine evento, ricerca con suggerimenti, filtri, salvataggio, vetrina, contatori. Nuovi capitoli tecnici "Mappe e tile", "Ricerca, filtri e stato nell'URL" e "Come si verifica una pagina". Roadmap rivista: reclamo in Fase 3, editor eventi in Fase 4. Corrette quattro indicazioni risultate imprecise: elenchi Artisti/Locali (sono segnaposto, non elenchi), protezione password compromesse (funzione del piano Pro), **player Spotify** (predisposto ma spento, non visibile) e **navigazione mobile** (a tendina dall'alto, non una barra fissa in basso) |
| 1.2 | 21 ottobre 2026 | 1 · 1.2 · 1.3 · 2 · 3 · 4 · 5.4 | Fasi 3 e 4: URL `/it` / `/en`, directory e schede pubbliche, reclamo, dashboard, editor eventi, pagina evento con calendario/condivisione, preferiti, qualità della posizione, serate collegate, segnalazioni, moderazione e serate passate. Fase 5.4: Home "muro di locandine" con Anton e IBM Plex Mono, lista a righe, mappa a tutta pagina, ticker, animazioni CSS e guscio aggiornato. Aggiornati struttura del progetto, variabili, tabelle, migrazioni, RLS, design system, roadmap e diario. Corrette le indicazioni ormai false su schede "segnaposto", creazione eventi assente, reclamo assente e preferiti senza pagina |
| 1.3 | 22 ottobre 2026 | 1 · 1.2 · 1.3 · 2 · 3 · 4 · 5.4 · 6 (6.1 → 6.3) | Fase 6: pagine pubbliche reali ( `/about`, `/per-gli-artisti`, `/per-le-venue`, `/contatti`, `/faq` ), indice e pagine città con soglia `noindex`, modulo contatti che scrive in database e coda messaggi admin. Aggiunte la sezione Città in Home (massimo sei città in ordine alfabetico) e l'evidenza del pulsante Dashboard. Aggiornati struttura del progetto, migration `0028`, RLS, roadmap e diario |
| 1.4 | 24 ottobre 2026 | 1 · 1.2 · 1.3 · 2 · 3 · 4 · 5.4 · 5.5A · 5.5B · 6 · 7A | Fase 5.5A: restyling login/registrazione/account/onboarding. Fase 7A: dominio, Aruba, Resend, SMTP Supabase, conferma email, reset password, Turnstile lato Supabase e callback OAuth. Fase 5.5B: scheda evento e popup/bottom sheet mappa. Aggiornati guida utente, manuale tecnico, diario, roadmap, variabili d'ambiente e risoluzione errori |
| 1.5 | 24 ottobre 2026 | 1 · 1.2 · 1.3 · 2 · 3 · 4 · 5.4 · 5.5A · 5.5B · 6 · 7A | Corretto il flusso email cross-browser: conferma account e reset password usano `token_hash` e `/auth/confirm`, quindi funzionano anche se il link viene aperto da Chrome/Gmail dopo una richiesta fatta in Opera. Aggiunto il controllo del ruolo al login per chi non è passato dal callback |
| 1.6 | 24 ottobre 2026 | 1 · 1.2 · 1.3 · 2 · 3 · 4 · 5.4 · 5.5A · 5.5B · 5.5C · 6 · 7A | Fase 5.5C: directory artisti/locali e schede con card tipografiche, monogrammi, filtri squadrati e metadati in righe. Aggiornati diario, roadmap e PDF |

## Come si aggiorna

1. Modifica o aggiungi i file `.md` in `docs/manuale/` (l'ordine dei capitoli segue il nome del file: `01-`, `02-`, …).
2. Aggiorna `versione` e `data` in `docs/manuale/meta.json` e aggiungi una riga a questa tabella.
3. Esegui `npm run manuale`: il PDF `docs/manuale/OpenStage-Manuale.pdf` viene rigenerato con indice e numeri di pagina aggiornati.
4. Committa i sorgenti **e** il PDF.